Flows, Energy Models and Score-Based Generation

Extended deep learning notes Sources: listed under References

What this file is really about:

$$\text{exact densities by changing variables} \rightarrow \text{densities up to a constant} \rightarrow \text{learn the score} \rightarrow \text{SDEs a}$$

How to use this file. Read after Diffusion in Depth. Part B has complete code, checked by running it. Part C is practice with answers.

Step-up path

You need. Diffusion in depth.

1. Change variables.
2. Build a flow.
3. Sample with energy and Langevin.
4. Match scores.
5. Use SDEs and one-step models.

You can stop when you can connect every family of generative model.

Contents

Part A Maths From the Ground Up

A1 The change of variables

Let $z \sim p_Z$ and $x = f(z)$, with f invertible. Probability in a small region is conserved, so

$$p_X(x) = p_Z(f^{-1}(x)) \left| \det \frac{\partial f^{-1}}{\partial x} \right|.$$

A **normalizing flow** stacks many invertible layers. The log-determinants add:

$$\log p(x) = \log p_Z(z) + \sum_k \log \left| \det \frac{\partial f_k^{-1}}{\partial x_k} \right|.$$

Unlike a VAE or a GAN, a flow gives an **exact** likelihood.

A2 Coupling layers

A general determinant costs $O(d^3)$. **RealNVP** splits the variables into a and b, keeps a unchanged, and scales and shifts b:

$$y_a = x_a, \qquad y_b = x_b \odot e^{s(x_a)} + t(x_a).$$

The Jacobian is triangular, so its log-determinant is just $\sum s(x_a)$. The inverse needs no inversion of s or t:

$$x_b = (y_b - t(y_a)) \odot e^{-s(y_a)}.$$

s and t can be any network. Alternate which half is transformed.

A3 Energy-based models

Define a density through any scalar function:

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}, \qquad Z_\theta = \int e^{-E_\theta(x)}\, dx.$$

The flexibility costs an intractable Z. The maximum-likelihood gradient is still simple:

$$\nabla_\theta \log p(x) = -\nabla_\theta E(x) + \mathbb{E}_{x' \sim p_\theta}\big[\nabla_\theta E(x')\big].$$

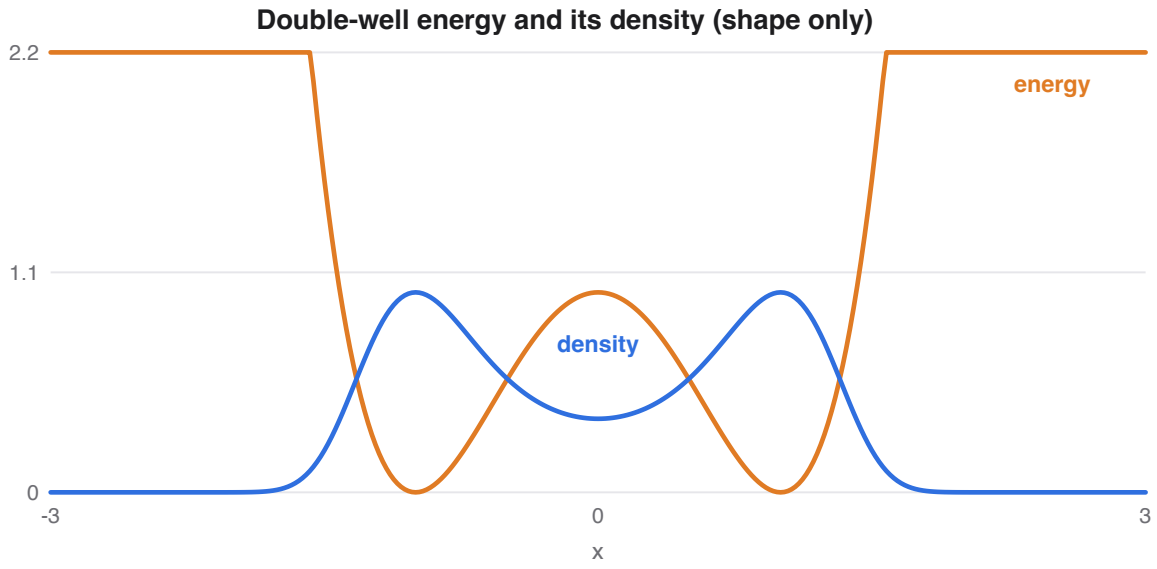
Lower the energy on data, and raise it on samples from the model. This is **contrastive divergence** when the samples come from a short chain.

A4 Langevin dynamics

To sample using only the gradient of the log density,

$$x_{k+1} = x_k + \eta\, \nabla \log p(x_k) + \sqrt{2\eta}\, \xi_k, \qquad \xi_k \sim \mathcal{N}(0, I).$$

This needs the **score** $\nabla \log p$, which does not depend on Z. As $\eta \to 0$ and the number of steps grows, samples approach p.



A5 Denoising score matching, derived

The score of the data is unknown. Add noise, $\tilde{x} = x + \sigma\varepsilon$. The conditional score is known exactly:

$$\nabla_{\tilde{x}} \log q(\tilde{x} \mid x) = -\frac{\tilde{x} - x}{\sigma^2} = -\frac{\varepsilon}{\sigma}.$$

Vincent's result is that regressing on this conditional score has the same minimiser as the true score of the noised distribution:

$$\mathbb{E}\left\| s_\theta(\tilde{x}) + \frac{\varepsilon}{\sigma} \right\|^2.$$

The optimal s averages over which clean x could have produced $\tilde{x}$.

A6 Diffusion as a stochastic differential equation

Letting the number of noise steps grow without limit gives a forward SDE. The **variance-preserving** version is

$$dx = -\tfrac{1}{2}\beta(t)\, x\, dt + \sqrt{\beta(t)}\, dW.$$

Anderson's theorem gives the reverse-time equation:

$$dx = \left[-\tfrac{1}{2}\beta x - \beta\, \nabla_x \log p_t(x) \right] dt + \sqrt{\beta}\, d\bar{W}.$$

The score is all that is needed.

A7 The probability flow ODE

A **deterministic** equation has the same marginal distributions at every time:

$$\frac{dx}{dt} = -\tfrac{1}{2}\beta x - \tfrac{1}{2}\beta\, \nabla_x \log p_t(x).$$

DDIM is one way of integrating it. A deterministic map also gives exact likelihoods, and reproducible noise-to-image inversions. Stochastic samplers tend to correct errors better, and deterministic samplers use fewer steps.

A8 Consistency models

Every point on an ODE trajectory should map to the same endpoint. A **consistency model** learns $f(x_\sigma, \sigma)$ with the property

$$f(x_\sigma, \sigma) = f(x_{\sigma'}, \sigma') \quad \text{on one trajectory}, \qquad f(x, \sigma_{\min}) = x.$$

The boundary condition is built in using $f = c_{\text{skip}}(\sigma)x + c_{\text{out}}(\sigma)F_\theta(x, \sigma)$. Training compares adjacent noise levels against a slowly updated target network. Once trained, sampling takes a single network evaluation.

A9 Comparing the families

Family	Exact likelihood	Sampling cost	Main weakness
Flow	Yes	One pass	Restricted architecture
VAE	Lower bound	One pass	Often blurry
GAN	No	One pass	Unstable training
Energy-based	Up to a constant	Long chains	Slow, hard to train
Diffusion	Through the ODE	Many steps	Slow sampling
Consistency	No	One or a few	Quality trade-off

Part B In Code

B1 A RealNVP flow

```
import torch
from torch import nn
from sklearn.datasets import make_moons

torch.manual_seed(0)
data = lambda n: torch.tensor(make_moons(n, noise=0.05, random_state=None)[0], dtype=torch.


class Coupling(nn.Module):
    def __init__(self, flip):
        super().__init__()
        self.flip = flip
        self.net = nn.Sequential(nn.Linear(1, 64), nn.ReLU(), nn.Linear(64, 64), nn.ReLU(),

    def forward(self, x):                                    # data towards noise
        a, b = x.flip(1).chunk(2, 1) if self.flip else x.chunk(2, 1)
        s, t = self.net(a).chunk(2, 1)
        s = torch.tanh(s) * 2
        y = torch.cat([a, b * s.exp() + t], 1)
        return (y.flip(1) if self.flip else y), s.sum(1)

    def inverse(self, y):
        a, b = y.flip(1).chunk(2, 1) if self.flip else y.chunk(2, 1)
        s, t = self.net(a).chunk(2, 1)
        s = torch.tanh(s) * 2
        x = torch.cat([a, (b - t) * (-s).exp()], 1)
        return x.flip(1) if self.flip else x


flow = nn.ModuleList(Coupling(i % 2 == 1) for i in range(8))
```

```
def log_prob(x):
    logdet = 0
    for f in flow:
        x, d = f(x)
        logdet = logdet + d
    return -0.5 * (x ** 2).sum(1) - torch.log(torch.tensor(2 * torch.pi)) + logdet


opt = torch.optim.Adam(flow.parameters(), lr=2e-3)
start = -log_prob(data(2000)).mean().item()
for step in range(2500):
    loss = -log_prob(data(256)).mean()
    opt.zero_grad(); loss.backward(); opt.step()

x = data(4)
z = x
for f in flow:
    z = f(z)[0]
back = z
for f in reversed(flow):
    back = f.inverse(back)

print('negative log-likelihood', round(start, 3), 'to', round(loss.item(), 3))
assert (back - x).abs().max() < 1e-3 and loss.item() < start - 1
```

B2 Langevin sampling from a known energy

```
import numpy as np

rng = np.random.default_rng(0)
score = lambda x: -4 * x * (x ** 2 - 1)            # energy (x^2 - 1)^2, with wells at -1
x = rng.normal(size=4000)
eta = 0.005
for _ in range(4000):
    x = x + eta * score(x) + np.sqrt(2 * eta) * rng.normal(size=x.shape)

grid = np.linspace(-3, 3, 20001)
p = np.exp(-(grid ** 2 - 1) ** 2); p /= p.sum()
second = (p * grid ** 2).sum()
print('sampled second moment', round((x ** 2).mean(), 3), 'exact', round(second, 3))
assert abs((x ** 2).mean() - second) < 0.05
```

B3 Training an energy-based model

```
import torch
from torch import nn

torch.manual_seed(0)
data = lambda n: torch.cat([torch.randn(n // 2, 1) * 0.3 - 2, torch.randn(n - n // 2, 1) *
E = nn.Sequential(nn.Linear(1, 64), nn.SiLU(), nn.Linear(64, 64), nn.SiLU(), nn.Linear(64,
opt = torch.optim.Adam(E.parameters(), lr=1e-3)
chains = torch.randn(512, 1) * 3                    # persistent chains


def langevin(x, steps=30, eta=0.01):
    for _ in range(steps):
        x.requires_grad_(True)
        g = torch.autograd.grad(E(x).sum(), x)[0]
        x = (x - eta * g + (2 * eta) ** 0.5 * torch.randn_like(x)).detach()
    return x


for step in range(1500):
    chains = langevin(chains)
    loss = E(data(512)).mean() - E(chains).mean() + 0.01 * (E(chains) ** 2).mean()
    opt.zero_grad(); loss.backward(); opt.step()

left = (chains < 0).float().mean().item()
near = ((chains.abs() - 2).abs() < 1).float().mean().item()
print('share on the left', round(left, 2), 'share near a mode', round(near, 2))
assert 0.15 < left < 0.85 and near > 0.7
```

B4 Denoising score matching against the exact score

```
import math
import torch
from torch import nn

torch.manual_seed(0)
sigma = 0.5
data = lambda n: (torch.rand(n, 1).round() * 4 - 2) + 0.2 * torch.randn(n, 1)
net = nn.Sequential(nn.Linear(1, 64), nn.SiLU(), nn.Linear(64, 64), nn.SiLU(), nn.Linear(64
opt = torch.optim.Adam(net.parameters(), lr=2e-3)

for step in range(4000):
    eps = torch.randn(512, 1)
    noisy = data(512) + sigma * eps
    loss = ((net(noisy) + eps / sigma) ** 2).mean()
    opt.zero_grad(); loss.backward(); opt.step()

v = 0.2 ** 2 + sigma ** 2


def exact(x):                                        # score of the noised two-peak mixture
    a, b = (-(x + 2) ** 2 / (2 * v)).exp(), (-(x - 2) ** 2 / (2 * v)).exp()
    return (a * -(x + 2) / v + b * -(x - 2) / v) / (a + b)


x = torch.linspace(-3, 3, 61).unsqueeze(1)
error = (net(x) - exact(x)).abs().mean().item()
print('mean error', round(error, 3), 'typical score size', round(exact(x).abs().mean().item
assert error < 0.15 * exact(x).abs().mean().item()
```

B5 Reverse SDE and probability flow

The data are Gaussian, so the score is exact, and only the integrators are tested.

```
import numpy as np

rng = np.random.default_rng(0)
m0, s0 = 2.0, 0.5
beta = lambda t: 0.1 + 19.9 * t
B = lambda t: 0.1 * t + 9.95 * t * t               # integral of beta from 0 to t


def score(x, t):
    a = np.exp(-0.5 * B(t))
    var = a * a * s0 * s0 + 1 - a * a
    return -(x - a * m0) / var


n, N = 20000, 1000
dt = 1.0 / N
sde = rng.normal(size=n); ode = sde.copy()
for k in range(N):
    t = 1 - k * dt
    b = beta(t)
    sde += dt * (0.5 * b * sde + b * score(sde, t)) + np.sqrt(b * dt) * rng.normal(size=n)
    ode += dt * (0.5 * b * ode + 0.5 * b * score(ode, t))

print('sde', sde.mean().round(3), sde.std().round(3), '| ode', ode.mean().round(3), ode.std
assert abs(sde.mean() - m0) < 0.05 and abs(sde.std() - s0) < 0.05
assert abs(ode.mean() - m0) < 0.05 and abs(ode.std() - s0) < 0.05
```

B6 A consistency model

```
import copy
import math
import torch
from torch import nn

torch.manual_seed(0)
m0, s0, s_min, s_max = 2.0, 0.5, 0.002, 20.0


def score(x, s):                                    # exact score of the noised data
    return -(x - m0) / (s0 ** 2 + s ** 2)


class C(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(2, 128), nn.SiLU(), nn.Linear(128, 128), nn.SiLU
    def forward(self, x, s):
        skip = 0.25 / ((s - s_min) ** 2 + 0.25)
        out = 0.5 * (s - s_min) / (0.25 + s ** 2).sqrt()
        return skip * x + out * self.net(torch.cat([x / (s ** 2 + 1).sqrt(), s.log() / 4],


online = C(); target = copy.deepcopy(online)
opt = torch.optim.Adam(online.parameters(), lr=1e-3)
N = 40
levels = (s_min ** (1 / 7) + torch.arange(N + 1) / N * (s_max ** (1 / 7) - s_min ** (1 / 7)

for step in range(4000):
    k = torch.randint(N, (256, 1))
    s1, s2 = levels[k], levels[k + 1]
    x2 = m0 + s0 * torch.randn(256, 1) + s2 * torch.randn(256, 1)
```

```
    x1 = x2 + (s1 - s2) * (-s2 * score(x2, s2))      # one Euler step of the ODE
    with torch.no_grad():
        y = target(x1, s1)
    loss = ((online(x2, s2) - y) ** 2).mean()
    opt.zero_grad(); loss.backward(); opt.step()
    with torch.no_grad():
        for p, q in zip(target.parameters(), online.parameters()):
            p.mul_(0.99).add_(0.01 * q)

with torch.no_grad():
    sample = target(s_max * torch.randn(20000, 1), torch.full((20000, 1), s_max))
print('one-step samples: mean', round(sample.mean().item(), 3), 'std', round(sample.std().i
assert abs(sample.mean() - m0) < 0.2 and abs(sample.std() - s0) < 0.2
```

Part C Practice

P1. $x = 2z$ with z standard normal. Find $p_X(0)$.

$p_Z(0) \times \frac{1}{2} = 0.399/2 = 0.199.$

P2. A coupling layer has $s = (0.5, -0.2)$. Find the log-determinant.

$0.3.$

P3. Why does Langevin dynamics not need the normalising constant?

$\nabla \log(e^{-E}/Z) = -\nabla E$, because Z does not depend on x.

P4. With $\sigma = 0.5$ and $\varepsilon = 1$, what is the regression target for the score?

$-\varepsilon/\sigma = -2.$

P5. Which has the same marginals as the reverse SDE, but no noise?

The probability flow ODE.